# Pokémon aizsardzības komandas optimizācija ar imitēto atkvēlināšanu

Praktiskā kombinatoriskā optimizācija | 2026. gada 11. oktobris
Students: Raivis Randohs
Repozitorijs: github.com/phantzzy/Pokemon-Team-Optimziation---PKO

## 1. Problēmas formulējums

Uzdevums ir no fiksētas N kandidātu kopas izvēlēties tieši sešus unikālus Pokémon. Modelī tiek vērtēti tikai aizsardzības tipi. Gājieni, statistikas rādītāji, spējas, priekšmeti, EV, IV, Tera tipi, lietojuma dati un cīņas simulācija netiek ņemti vērā.

Ar P apzīmē kandidātu kopu, bet ar T - risinājumu. Derīgs risinājums ir nesakārtota sešu elementu P apakškopa. Meklēšanas telpā ir C(N, 6) komandas: no 924, ja N=12, līdz 1 192 052 400, ja N=100.

Uzbrūkošajam tipam t un Pokémon p ar m(t,p) apzīmē standarta tipa efektivitātes reizinātāju aizsardzībā. Divu tipu efektivitāte ir abu atsevišķo tipu reizinātāju reizinājums.

> **minimizēt f(T) = summa visiem p no T un t no tipu kopas: m(t,p)**

Mazāka izmaksu vērtība nozīmē vidēji labāku aizsardzību. Interpretācijai izmanto VidējaisReizinātājs = f(T) / 108, jo seši Pokémon tiek vērtēti pret 18 uzbrukuma tipiem.

## 2. Izstrādātais risinājums

### a) Risinājumu apgabals

Risinājums satur sešus unikālus kandidātus. Piecās fiksētās, savstarpēji ligzdotās instancēs ir 12, 20, 30, 50 un 100 Pokémon. Lokālajā datu kopā ir 100 sugas un pilna 18 reiz 18 tipu tabula no PokéAPI. Tiek pārbaudīta piederība instancei, izmērs un unikalitāte.

| Instance | N | Iespējamās komandas |
|---|---|---|
| small_12 | 12 | 924 |
| medium_20 | 20 | 38 760 |
| medium_30 | 30 | 593 775 |
| large_50 | 50 | 15 890 700 |
| xlarge_100 | 100 | 1 192 052 400 |

### b) Novērtēšana

Katram Pokémon iepriekš aprēķina 18 aizsardzības reizinātāju vektoru. Komandas novērtējums ir sešu vektoru summa. Aprēķins ir deterministisks, nav atkarīgs no komandas secības un optimizācijas ciklā nelasa failus.

# 2. Izstrādātais risinājums - turpinājums

## c) Gājiens un apkārtne

Kaimiņrisinājumu iegūst, vienu pašreizējās komandas dalībnieku vienmērīgi nejauši izvēloties izņemšanai un vienu ārpus komandas esošu kandidātu - pievienošanai. Rezultāts vienmēr ir derīga sešu elementu komanda bez dublikātiem.

## d) Imitētā atkvēlināšana

Imitētās atkvēlināšanas (SA) algoritms ir izstrādāts manuāli, neizmantojot optimizācijas bibliotēku. Tas sāk darbu ar nejaušu komandu, izmantojot noteiktu sēklas vērtību. Delta ir kandidāta un pašreizējā risinājuma izmaksu starpība. Labāku vai vienādu kandidātu vienmēr pieņem, bet sliktāku pieņem ar varbūtību:

**P(pieņemt) = exp(-delta / temperatūra)**

Pēc katras iterācijas temperatūru reizina ar alpha. Pašreizējais risinājums un visā izpildē atrastais labākais risinājums tiek glabāti atsevišķi. Izpilde beidzas pie iterāciju ierobežojuma vai tad, kad temperatūra nokrīt zem minimuma.

Noklusētie parametri ir sākuma temperatūra 10,0, alpha 0,995, minimālā temperatūra 0,0001 un ne vairāk kā 20 000 iterāciju. Temperatūras nosacījums noklusēto izpildi aptur pēc 2297 iterācijām. Vienāda instance, sēkla un parametri reproducē vienādu rezultātu.

Atsevišķs precīzais risinātājs mazām instancēm pārbauda visas sešu elementu kombinācijas. Tas pierāda salīdzināšanai izmantoto optimumu, bet nav galvenā optimizācijas metode.

# 3. Testēšanas metodika

Projektā ir 48 automatizēti vienību un integrācijas testi. Tie pārbauda tipu mijiedarbības, datu validāciju, deterministisku novērtēšanu, apkārtnes invariantus, algoritma derīgumu un reproducējamību, apstāšanās nosacījumus, precīzā risinātāja korektumu, fiksētās instances, komandrindas saskarni, novirzes un CSV izvadi. Visi testi sekmīgi izpildīti ar Python 3.11.9.

Algoritms katrai instancei tika izpildīts 30 reizes ar sēklām no 0 līdz 29. Katrai izpildei reģistrēja izmaksas, vidējo reizinātāju, laiku, sēklu un komandu. Instances līdz N=30 atrisināja arī precīzi. Novirzi aprēķināja šādi:

**Novirze(%) = (SA - OPT) / OPT * 100**

Ja N=50 un N=100, pilnā pārlase netika veikta. Mazākā no visās algoritma izpildēs novērotajām izmaksām ir labākā zināmā vērtība, nevis pierādīts optimums.

## Pierādītie precīzie rezultāti

| Instance | Pārbaudītās komandas | Optimālās izmaksas |
|---|---|---|
| small_12 | 924 | 108,75 |
| medium_20 | 38 760 | 104,50 |
| medium_30 | 593 775 | 103,50 |

# 4. Etalonmērījumu rezultāti

| Instance | Atsauce | Labākā | Vidējā | Sliktākā | Vid. novirze | Vid. laiks |
|---|---|---|---|---|---|---|
| small_12 | 108,75 OPT | 108,75 | 108,75 | 108,75 | 0,0000% | 9,418 ms |
| medium_20 | 104,50 OPT | 104,50 | 104,50 | 104,50 | 0,0000% | 11,242 ms |
| medium_30 | 103,50 OPT | 103,50 | 103,50 | 103,50 | 0,0000% | 12,652 ms |
| large_50 | 101,75 LZ | 101,75 | 101,75 | 101,75 | 0,0000% | 14,534 ms |
| xlarge_100 | 93,75 LZ | 93,75 | 93,75 | 93,75 | 0,0000% | 24,019 ms |

OPT = pierādīts optimums; LZ = algoritma novērotā labākā zināmā vērtība.

## Interpretācija

Visās 30 izpildes reizēs algoritms sasniedza pierādīto optimumu instancēm ar 12, 20 un 30 kandidātiem. Tādēļ to labākā, vidējā un sliktākā novirze bija nulle. Arī N=50 un N=100 gadījumā visas izpildes atgrieza vienādas izmaksas. Šīs vērtības paliek tikai labākās zināmās, jo pilnā pārlase netika veikta.

Vidējais izpildes laiks pieauga no 9,418 ms, ja N=12, līdz 24,019 ms, ja N=100, tādēļ tas ir piemērots interaktīvai lietošanai. Stabilitāte ir ticama šim vienkāršajam aditīvajam aizsardzības izmaksu modelim, bet nepierāda abu lielo instanču globālo optimalitāti.

# 5. Secinājumi

Projekts demonstrē pilnu optimizācijas darba procesu: precīzu risinājumu apgabalu, deterministisku mērķfunkciju, derīgumu saglabājošu apkārtni, manuāli izstrādātu imitētās atkvēlināšanas algoritmu, precīzu pārbaudi mazām instancēm, reproducējamus eksperimentus un CSV rezultātus. Realizācija droši atrada visus zināmos mazo instanču optimumus un stabilus labākos zināmos rezultātus lielākām meklēšanas telpām.

Datu avots: lokāls PokéAPI momentuzņēmums. Pilns pirmkods, instances, testi un etalonmērījumu CSV ir pieejami iepriekš norādītajā repozitorijā.